

GSAP

GreenSock Animation Platform

Beginner se Expert · Hinglish Handwritten Notes —■

Ch 1 — GSAP Kya Hai? Setup & Philosophy
Ch 2 — `gsap.to()` Deep Dive
Ch 3 — `gsap.from()` & `gsap.fromTo()`
Ch 4 — `gsap.set()` & Immediate Renders
Ch 5 — Easing — Animation ka Soul
Ch 6 — Timeline — Sequencing Master
Ch 7 — Stagger — Multiple Elements
Ch 8 — ScrollTrigger — Scroll Magic
Ch 9 — Callbacks & Control Methods
Ch 10 — Plugins Deep Dive
Ch 11 — Performance & GPU Tricks
Ch 12 — React / Vue Integration
Ch 13 — Real Projects + Exercises

Made with ♥■ for Anand · 2025

Chapter 1 — GSAP Kya Hai? ■

GSAP = GreenSock Animation Platform

Ek professional-grade JavaScript library hai jo web elements ko smoothly, precisely, aur performantly animate karta hai.

■ REAL LIFE ANALOGY:

Socho tum ek film director ho. Tumhare paas actors (DOM elements) hain.

CSS animation = sirf ek actor ko bola 'jao wahan' — koi timing control nahi.

GSAP = poor direction system: kab jaana, kaise jaana, kab rukna,

uska actor kab start kare, ek saath karo ya sequence mein — sab control!

Matlab GSAP tumhara animation ka DIRECTOR hai. ■

■ CSS Animation vs GSAP — Ek Honest Comparison

Feature	CSS Animation / Transition	GSAP	Winner
Performance	Good (GPU pe kuch)	Blazing fast — ALWAYS GPU	GSAP
JS Integration	Hacky — event listeners	Native JS — full control	GSAP
Sequencing	Delay hack se karo	Timeline — built-in	GSAP
Dynamic Values	Nahi — CSS static hai	Runtime mein change karo	GSAP
Reverse / Pause	Nahi / hacky	tl.pause(), tl.reverse()	GSAP
Scroll Animation	Limited (@scroll-timeline)	ScrollTrigger = magic	GSAP
Cross Browser	Prefixes lagao	Auto handles	GSAP
Learning Curve	Easy but limited	Medium — worthy	Depends
File Size	Zero (built-in)	~30KB minzipped	CSS
SVG Animation	Very limited	MorphSVG, DrawSVG, path	GSAP
Physics / Spring	Nahi	Elastic, bounce, inertia	GSAP
3D Transforms	CSS 3D lagana padta	Full 3D built-in	GSAP

Setup — 3 Tarike

●●● Method 1 — CDN

```
// METHOD 1: CDN (Recommended for beginners)
// HTML ke <head> ya <body> end mein lagao:
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js"></script>

// Koi import nahi — seedha gsap.to() use karo!
```

●●● Method 2 — NPM

```
// ■■■ METHOD 2: NPM (Modern projects – React, Vue, Vite) ■■■■  
npm install gsap  
  
// Phir JS file mein import karo:  
import { gsap } from 'gsap';  
  
// Plugin bhi import karo agar chahiye:  
import { ScrollTrigger } from 'gsap/ScrollTrigger';  
gsap.registerPlugin(ScrollTrigger); // MUST register before use!
```

● ● ● *Method 3 — Club Plugins*

[illegible]

■■ IMPORTANT — registerPlugin:

`qsap.registerPlugin()` MUST call karo plugins ke liye — warna kaam nahi karega!

Multiple plugins ek call mein: `gsap.registerPlugin(ScrollTrigger, Flip, Draggable);`

multiple plugins ek hi mein: `gsap.registerPlugin(CssPlugin, Flip, Draggable)`
 registerPlugin ko app ke start mein ek baar call karo — baar baar mat karo.

■ GSAP Internally Kaise Kaam Karta Hai?

GSAP CSS properties directly nahi change karta — ye element ka style object modify karta hai directly, aur requestAnimationFrame (rAF) use karta hai frame-by-frame update ke liye.

Internally GSAP ek Tween object banata hai jisme hota hai:

- target element (ya elements ka array)
- start values (current state se capture karta hai)
- end values (tum jo dete ho)

Chapter 2 — gsap.to() ■

Definition: Element ko uski CURRENT state se TARGET state pe animate karo.

Matlab: element jahan bhi hai, wahan se => tumhare diye hue values tak jaata hai.

Syntax: gsap.to(target, varsObject)

■ REAL LIFE ANALOGY:

Delhi se Mumbai jaana: tum Delhi pe ho (current state), Mumbai tumhara target.

`gsap.to('.box', { x: 500 })` => box jahan bhi hai wahan se x=500 tak jaata hai.

Tum nahi batate ki Delhi kahan hai — GSAP khud detect karta hai current position!

■ Syntax ka Poora Breakdown

● ● ● target — kisko animate karein?

● ● ● *varsObject* — *poori property list with comments*

```
// ■■■ varsObject ke andar kya daal sakte ho ■■■■
```

```

gsap.to('.box', {
  // TRANSFORM properties (GPU accelerated – use these!)
  x: 200,           // translateX – pixels mein (CSS: transform: translateX(200px))
  y: -100,          // translateY – negative = upar
  xPercent: -50,    // translateX in % of element's own width
  yPercent: -50,    // translateY in % of element's own height
  rotation: 360,    // degrees – clockwise
  rotationX: 45,    // 3D X-axis rotation (needs perspective on parent)
  rotationY: 180,   // 3D Y-axis rotation (card flip effect!)
  rotationZ: 45,    // same as rotation
  scale: 1.5,       // uniform scale (1 = normal, 2 = double size)
  scaleX: 2,        // sirf horizontal stretch
  scaleY: 0.5,      // sirf vertical squeeze
  skewX: 15,        // horizontal skew (slanted look)
  skewY: 10,        // vertical skew

  // APPEARANCE
  opacity: 0,       // 0 = invisible, 1 = fully visible
  autoAlpha: 0,     // opacity + visibility: hidden jab 0 ho (BETTER than opacity!)

  // CSS PROPERTIES (snake-case ko camelCase mein likho)
  backgroundColor: "#ff0000",
  borderRadius: "50%",
  width: 200,       // number = pixels, string = '200px' ya '50%'
  height: 100,
  fontSize: 24,
  lineHeight: 1.5,
  color: 'blue',    // text color
  padding: 20,
  margin: 0,

  // TIMING & CONTROL
  duration: 1.5,    // seconds mein (default = 0.5s)
  delay: 0.3,       // animation start hone se pehle kitna wait
  ease: "power2.out", // animation curve (next chapter mein detail)
  repeat: 2,        // kitni baar repeat (-1 = infinite loop)
  repeatDelay: 0.5, // har repeat ke beech gap
  yoyo: true,       // forward + reverse alternating (like yoyo toy!)
  yoyoEase: 'power1.in', // reverse direction ka alag ease
  stagger: 0.1,     // multiple elements ke beech delay (Ch 7 mein detail)
  overwrite: 'auto', // conflicting tweens handle karo

```

```
onStart: () => console.log('Animation shuru!'),
```

```
onUpdate: () => console.log('Har frame pe!'),
```

```
onComplete: () => console.log('Khatam!'),
```

■ autoAlpha — Opacity ka Intelligent Version

autoAlpha opacity aur visibility dono ek saath handle karta hai.

Jab value 0 hoti hai, visibility: hidden bhi set hota hai — screen readers ke liye better!

Jab value 1 hoti hai, visibility: visible automatically set hota hai.

```
●●● autoAlpha vs opacity
// BAD — opacity: 0 pe tab bhi element space leta hai
// aur screen readers usse still read kar sakte hain
gsap.to('.modal', { opacity: 0, duration: 0.3 });

// GOOD — autoAlpha sab handle karta hai
gsap.to('.modal', { autoAlpha: 0, duration: 0.3 });
// jab 0 hoga => opacity:0 + visibility:hidden
// jab 1 hoga => opacity:1 + visibility:visible (auto!)

// Extra: agar element initially CSS mein visibility:hidden hai
// aur tum gsap.from('.el', { autoAlpha: 0 }) karo
// GSAP visibility:visible bhi set kar deta hai — helpful!
```

■ overwrite — Conflicting Animations Handle Karna

Jab ek hi element pe multiple tweens chal rahe ho, conflict hoti hai.

overwrite property batati hai ki purani animation ka kya karo:

```
●●● overwrite options
// overwrite ke options:

// false (default) — dono tweens challenge, conflict hogi (bad!)
gsap.to('.box', { x: 200, overwrite: false });

// true — us element ke SAARE purane tweens kill kar do
gsap.to('.box', { x: 200, overwrite: true });

// 'auto' (RECOMMENDED) — sirf conflicting properties kill karo
// e.g., agar purana tween x animate kar raha tha,
// aur naya bhi x animate kare, toh purana x sirf kill hoga
// lekin purana opacity chal■■■■■ — smart!
gsap.to('.box', { x: 200, overwrite: 'auto' });

// Global default set karo (recommended):
gsap.defaults({ overwrite: 'auto' }); // app start mein ek baar
```

■ COMMON MISTAKE:

MISTAKE: button hover pe gsap.to() call karte rahna without overwrite.

Result: Ek baar hover, phir quickly another hover — 2 tweens conflict mein!

FIX: gsap.to('.btn', { scale: 1.2, overwrite: 'auto' });

MISTAKE: x/y ke saath left/top bhi animate karna.

Result: Dono fight karte hain — animation glitch hoti hai.

FIX: Sirf ek use karo — hamesha x/y prefer karo (faster).

■ gsap.to() Return Value — Tween Object

```
●●● Tween object ke methods
// gsap.to() ek Tween object return karta hai — isse save karo!
const tween = gsap.to('.box', { x: 200, duration: 1, paused: true });

// Tween ke methods:
tween.play();           // start / resume
tween.pause();          // pause karo
tween.reverse();        // ulta chalao
tween.restart();        // beginning se shuru
tween.seek(0.5);        // 0.5 second ke point pe jaao
tween.progress(0.75);   // 75% progress pe jaao (0 to 1)
tween.timeScale(2);     // 2x speed
tween.timeScale(0.5);   // slow motion!
tween.kill();           // permanently destroy
tween.invalidate();     // start values recapture karo (re-run ke liye)

// Getters:
tween.duration();       // total duration
tween.time();           // current time
tween.progress();       // current progress (0-1)
tween.isActive();       // kya currently chal raha hai?
tween.paused();         // kya paused hai?
```

■ Practical Examples

●●● Practical examples

```
// 1. Simple move + fade
gsap.to('.hero', { x: 100, opacity: 0.5, duration: 1.5 });

// 2. Infinite spinning loader
gsap.to('.loader', { rotation: 360, duration: 1, repeat: -1, ease: 'none' });

// 3. Button hover effect (play/reverse pattern)
const btnTween = gsap.to('.btn', { scale: 1.1, duration: 0.2, paused: true });
btn.addEventListener('mouseenter', () => btnTween.play());
btn.addEventListener('mouseleave', () => btnTween.reverse());

// 4. Counter animation (non-DOM object!)
const obj = { val: 0 };
gsap.to(obj, {
  val: 1000,
  duration: 2,
  onUpdate: () => {
    counter.textContent = Math.round(obj.val); // live update
  }
});

// 5. Card 3D flip
gsap.to('.card', { rotationY: 180, duration: 0.6, ease: 'power2.inOut' });

// 6. Pulse effect (yoyo infinite)
gsap.to('.dot', { scale: 1.3, opacity: 0.7, yoyo: true, repeat: -1, duration: 0.8 });

// 7. Multiple elements stagger (preview – detail Ch7 mein)
gsap.to('.card', { y: -20, opacity: 1, stagger: 0.1, duration: 0.5 });
```

Chapter 3 — gsap.from() & gsap.fromTo() ■

gsap.from() — 'Kahan se aao' wali animation

Element ki GIVEN values se uski CURRENT (natural) state pe animate karo.
Yani: element pehle invisible/offscreen DIKHA DETA HAI, phir apni jagah aata hai.

■ REAL LIFE ANALOGY:

Cricket match mein fielder boundary pe khada hai (given state = x: 500).
Ball aane pe wo wicket ki taraf dauda aata hai (current state = x: 0).
gsap.from('.fielder', { x: 500, opacity: 0 }) => boundary se wicket tak aata hai!
Tum batate ho KAHAN SE — GSAP KHUD jaanta hai kahan JAANA hai (current position).

●●● gsap.from() examples

```
// Syntax — bilkul gsap.to() jaisi, bas direction ulti hai
gsap.from( target, varsObject );

// Example 1: Hero heading upar se slide down karo
gsap.from('.hero h1', {
  y: -80,          // upar se aayega (y:-80 = 80px upar)
  opacity: 0,      // invisible se visible hoga
  duration: 1,
  ease: 'power3.out'
});

// Example 2: Left se slide in — nav links
gsap.from('.nav-link', {
  x: -60,
  opacity: 0,
  stagger: 0.1,    // har link 0.1s baad aaye
  duration: 0.6
});

// Example 3: Scale se pop in
gsap.from('.card', {
  scale: 0,
  opacity: 0,
  duration: 0.5,
  ease: 'back.out(1.7)' // slight overshoot — satisfying pop!
});

// Example 4: Page load sequence
gsap.from('.logo', { y: -30, opacity: 0, duration: 0.8 });
gsap.from('.nav', { y: -20, opacity: 0, duration: 0.8, delay: 0.2 });
gsap.from('.hero-text', { y: 40, opacity: 0, duration: 1, delay: 0.5 });
gsap.from('.cta-btn', { scale: 0.8, opacity: 0, duration: 0.6, delay: 0.9 });
```

■ COMMON MISTAKE:

MISTAKE: gsap.from() ke baad element wapas CURRENT state pe return aata hai — HAMESHA.
Agar tum chahte ho element permanently invisible rahe — gsap.from() galat tool hai!
FIX: Agar end state permanent chahiye => gsap.fromTo() use karo, ya gsap.to() use karo.

MISTAKE: Yeh sochna ki gsap.from() 'se' aata hai yani end bhi wahi hota hai.
REALITY: End hamesha CURRENT CSS state hoti hai. from() sirf start define karta hai.

■ gsap.fromTo() — Full Control (Start AND End dono define)

gsap.fromTo(target, fromVars, toVars)

Tum DONO states control karte ho — start bhi, end bhi.
CSS pe koi dependency nahi — 100% predictable.

■ REAL LIFE ANALOGY:

Train A se B tak: tum batate ho ki A kahan hai (fromVars) aur B kahan hai (toVars).
GSAP ko koi guess nahi karna padta — dono clearly define hain.
Best when: Page reload pe bhi same animation, ya when CSS state unreliable ho.

●●● gsap.fromTo() examples

```
// Syntax
gsap.fromTo( target, { FROM properties }, { TO properties + timing } );
//           |                               |
//           start state                     end state + duration/ease etc

// Example 1: Left se right tak — complete control
gsap.fromTo('.box',
  { x: -300, opacity: 0 }, // FROM: yahan se shuru karo
```

■ to vs from vs fromTo — Kab Kya Use Karo?

Method	Start State	End State	Best Use Case
gsap.to()	Auto (current CSS)	Tum define karo	Existing element ko kisi jagah le jao
gsap.from()	Tum define karo	Auto (current CSS)	Entrance / intro animations
gsap.fromTo()	Tum define karo	Tum define karo	Full control, reusable, reliable
gsap.set()	—	Tum define karo	Instant set, no animation

■ PRO TIP:

Rule of thumb: Page load entrance animations ke liye gsap.from() use karo.
ScrollTrigger animations ke liye gsap.fromTo() use karo — consistent results.
Button/interaction animations ke liye gsap.to() use karo.

Chapter 4 — gsap.set() ■ Instant Set

gsap.set() = gsap.to() with duration: 0 (Instant, koi animation nahi)

Element ki properties instantly set karo — bina animation ke.
Use karo: initial state setup ke liye, ya reset ke liye.

●●● gsap.set() use cases

```
// gsap.set() — simple but powerful
gsap.set('.box', { x: 100, opacity: 0, scale: 0.5 });

// Ye exactly same hai:
gsap.to('.box', { x: 100, opacity: 0, scale: 0.5, duration: 0 });

// When to use gsap.set():

// 1. Page load pe initial state set karo (FOUC prevent karo)
gsap.set('.card', { opacity: 0, y: 30 }); // pehle hide karo
// ...phir ScrollTrigger ya kuch se animate karo

// 2. Animation ke baad reset
gsap.to('.box', {
  x: 500, duration: 1,
  onComplete: () => gsap.set('.box', { x: 0 }) // wapas original pe
});

// 3. Multiple elements instantly same state pe
gsap.set('.nav-item, .hero, .card', { clearProps: 'all' }); // sab clear!

// 4. 3D setup karo pehle
gsap.set('.scene', { perspective: 800 }); // parent pe perspective
gsap.set('.card-3d', { transformStyle: 'preserve-3d' });

// clearProps — GSAP ke lagaye hue sab styles hata do
gsap.set('.el', { clearProps: 'all' }); // sab
gsap.set('.el', { clearProps: 'x,opacity' }); // specific
```

■ PRO TIP:

GSAP se set ki hui properties ko CSS override nahi karta agar inline style hai.
clearProps use karo jab tum chahte ho ki CSS apni values wapas le le.
Useful example: Animation end pe CSS class add karo aur gsap clearProps karo.

Chapter 5 — Easing ■ Animation Ka Soul

Ease = Animation ki speed curve — kaise chalta hai, kaise rukta hai

Bina ease ke animation mechanical lagti hai — robot ki tarah.

Sahi ease lagao aur animation zinda ho jaati hai — natural feel!

Analogy: Car brake karte waqt suddenly nahi rukti — dheere dheere rukti hai. Wahi feel!

■ 3 Ease Directions — .in, .out, .inOut

.in Slow start, fast end

Jaise gadi accelerate karti hai — pehle dheere, baad mein fast.

.out Fast start, slow end (MOST NATURAL)

Jaise ball throw ki — fast nikli, dheere aakar ruki. Best for entrances!

.inOut Slow start, fast middle, slow end

Jaise metro train — station pe slow, beech mein fast, station pe slow.

●●● Ease syntax

```
// Ease syntax
gsap.to(".box", { x: 200, ease: "power2.out" });
//           |           |
//           ease name   direction (.in/.out/.inOut)

// Shorthand — sirf name likhne pe .out apply hota hai by default:
gsap.to(".el", { x: 200, ease: "power2" }); // same as "power2.out"
```

■ Ease Types — Poori Dictionary

none / linear

Bilkul seedhi line — constant speed. Koi acceleration nahi.

Use: Rotating loader, progress bars, conveyor belt animation.

Syntax: "none" ya "linear"

power1

Gentle curve. Subtle ease — barely noticeable acceleration.

Use: Text fade-in, subtle hover effects.

Syntax: "power1.out" (default GSAP ease!)

power2

Moderate ease. 2nd degree polynomial curve. Very natural feel.

Use: Most UI animations — cards, modals, dropdowns.

Syntax: "power2.out" ← MOST USED

power3

Strong ease. Dramatic difference between start & end speed.

Use: Hero text reveal, large element entrances.

Syntax: "power3.out"

power4

Very dramatic. Almost instant start, very smooth stop (or vice versa).

Use: Impactful reveals, premium feel animations.

Syntax: "power4.out"

back

Overshoot karta hai — thoda aage jaata hai phir wapas aata hai. Springy!

Use: Button pop, element appear, playful UI elements.

Syntax: "back.out(1.7)" — 1.7 = overshoot amount

elastic

Spring jaisa — multiple bounces with decreasing amplitude.

Use: Notification pop, game elements, cartoon UI.

Syntax: "elastic.out(1, 0.3)" — amplitude, period

bounce

Ball girne jaisa — floor pe bounce karta hai.

Use: Score drop, item land effect, gravity simulation.

Syntax: "bounce.out"

circ

Circular arc curve — very smooth acceleration.

Use: Smooth modal open/close, refined UI.

Syntax: "circ.inOut"

expo

Exponential — extreme version of power. Very fast then very slow.

Use: Premium reveals, dramatic transitions.

Syntax: "expo.out"

sine

Smooth sinusoidal curve. Gentle, very natural.

Use: Breathing animations, subtle pulsing.

Syntax: "sine.inOut"

steps(n)

Stepwise — n steps mein jump karta hai. Bina smooth curve ke.

Use: Sprite sheet animation, typewriter, video game feel.

Syntax: "steps(5)" — 5 frames mein jaavega

slow(center, edges, ease)

Middle mein slow, edges pe fast. Speed modulate karo.

Use: Spotlight effect, reading highlight.

Syntax: "slow(0.7, 0.7, false)"

CustomEase

Khud Bezier curve banao! Plugin hai — import karna padega.

Use: Completely custom feel — no limits.

Syntax: CustomEase.create("hop", "M0,0 C0.5,0 0.5,1 1,1")

CustomBounce

Physics-based bounce customize karo (Club plugin).

Use: Realistic physics feel.

Syntax: CustomBounce.create("myBounce", { strength: 0.7, squash: 0.5 })

RoughEase

Random jitter/vibration effect. Rough, shaky feel.

Use: Earthquake, broken TV, nervous animation.

Syntax: "rough({ strength: 2, points: 20 })"

■ GSAP EASE VISUALIZER:

Ease Visualizer Tool: gsap.com/ease-visualizer — browser mein open karo, har ease ka curve dekho, interactive feel karo, code copy karo!

Best learning tool for easing — 10 minutes spend karo wahan.

■ Ease Shorthand Rules (important!)

● ● ● Ease shorthand rules

```
// RULE 1: Direction default = .out (most natural)
gsap.to(".el", { ease: "power2" }) // = "power2.out"
gsap.to(".el", { ease: "back" }) // = "back.out"
gsap.to(".el", { ease: "elastic" }) // = "elastic.out"

// RULE 2: ease: 'none' aur ease: 'linear' SAME hain
gsap.to(".el", { ease: "none" }) // constant speed
gsap.to(".el", { ease: "linear" }) // same thing

// RULE 3: Parametric eases — parentheses mein value do
gsap.to(".el", { ease: "back.out(2.5)" }) // more overshoot
gsap.to(".el", { ease: "elastic.out(1,0.3)" })
gsap.to(".el", { ease: "steps(8)" }) // 8 steps
gsap.to(".el", { ease: "slow(0.7, 0.7)" })

// RULE 4: Global default ease set karo
gsap.defaults({ ease: 'power2.out', duration: 0.8 });
// Ab har tween ka default ease yahi hoga

// RULE 5: Timeline ka default ease — sab tweens ko milega
const tl = gsap.timeline({ defaults: { ease: 'power3.out', duration: 1 } });
```


○

○

○

○

○

○

○

○

○

1

1

■ Real-World Timeline Example — Page Intro Animation

● ● ● Real-world example

```
// Complete landing page intro animation
const intro = gsap.timeline({
  defaults: { duration: 0.8, ease: 'power3.out' },
  delay: 0.3 // page load ke baad thoda wait
});

// Initial state set karo PEHLE
gsap.set(['.logo', '.nav', '.hero-title', '.hero-sub', '.hero-btn', '.hero-img'], {
  autoAlpha: 0
});

// Sequence
intro
.to(".logo", { autoAlpha: 1, y: 0, from: { y: -20 } })
.to(".nav a", { autoAlpha: 1, y: 0, stagger: 0.08 }, "<+0.2")
.to(".hero-title", { autoAlpha: 1, y: 0 }, "+=0.1")
.to(".hero-sub", { autoAlpha: 1, y: 0 }, "<+0.3")
.to(".hero-btn", { autoAlpha: 1, scale: 1, ease: "back.out(1.5)" }, "<+0.3")
.to(".hero-img", { autoAlpha: 1, x: 0, duration: 1 }, 0.5);
// hero-img simultaneously starts at 0.5s mark — overlap with text!
```

■ COMMON MISTAKE:

MISTAKE: Timeline mein individual tween ki duration global defaults override nahi karta. Agar tween mein { duration: 2 } diya aur defaults mein { duration: 1 } hai, tween ka OWN duration (2) win karta hai — individual > defaults.

MISTAKE: tl.to() ke return value ko chain karte karte bhool jaana position add karna. Always third argument check karo jab tweens overlap karne chahiye.

Chapter 7 — Stagger ■ Multiple Elements Magic

Stagger = Multiple elements ek ke baad ek animate karo — thoda gap ke saath

Bina stagger ke: sab elements ek saath animate hote hain (boring, blocky).

Stagger se: wave effect, cascade, domino — beautiful choreography!

Short: stagger: 0.1 ya stagger: { each: 0.1, from: 'center' }

■ REAL LIFE ANALOGY:

Domino toppling: pehla domino girta hai, phir 0.2s baad doosra, phir teesra...

Ya sochno army parade: soldiers ek ke baad ek qadam uthate hain, saath nahi.

```
gsap.to('.soldier', { y: -20, stagger: 0.1 }) => wave effect!
```

■ Stagger — Simple to Advanced

● ● ● *Stagger — simple to advanced*

```
// ■■■ SIMPLE STAGGER – just a number
```

```
gsap.from('.card', {
  opacity: 0,
  y: 40,
  stagger: 0.15, // har element 0.15s baad start karega
  duration: 0.6
});
```

```
// ■■■ NEGATIVE STAGGER – reverse order
```

```
gsap.from('.item', {
  opacity: 0,
  stagger: -0.1,    // last element PEHLE, phir upar ki taraf
  duration: 0.5
});
```

```
// ■■■ ADVANCED STAGGER OBJECT
```

```
gsap.to('.dot', {
  scale: 2,
  stagger: {
    each: 0.05,           // har element ke beech gap (seconds)
    amount: 1,           // TOTAL time for all staggers (vs 'each')
    //  each ya amount – ek time mein ek hi use karo!

    from: 'center',       // kahan se shuru ho (detail niche)

    grid: [4, 5],         // 2D grid – rows x columns
    //  ya grid: 'auto' – GSAP khud count kare

    axis: 'x',            // grid mein axis direction
    ease: 'power2.inOut', // stagger ka apna ease curve
  },
  duration: 0.4
});
```

■ stagger.from — Kahan Se Shuru?

```
from: "start"
```

Pehla element (index 0) pehle. Default. Left-to-right, top-to-bottom.

```
from: "end"
```

Last element pehle. Reverse order mein animate hoga.

```
from: "center"
```

Middle wala element pehle, phir bahar ki taraf wave. Beautiful!

```
from: "edges"
```

Dono ends pehle, phir center ki taraf. Closing effect.

```
from: "random"
```

Random order mein. Chaotic but interesting effect.

from: "0 / 1 / n"

Specific index wala element pehle. Usse bahar ki taraf wave.

from: "[0,2]"

Array of indices — multiple starting points!

■ **each vs amount** — Difference Samjho

● ● ● *each vs amount*

```
// ■■ each ■■ har element ke beech gap (regardless of count)
gsap.to('.item', { stagger: { each: 0.1 } });
// 5 elements => total stagger time = 0.1 * 4 = 0.4s
// 50 elements => total stagger time = 0.1 * 49 = 4.9s (much longer!)

// ■■ amount ■■ TOTAL stagger time fix karo
gsap.to('.item', { stagger: { amount: 1 } });
// 5 elements => each element 0.2s gap (1 / (5-1))
// 50 elements => each element 0.02s gap (1 / (50-1)) — stays 1s total!

// RULE: Use 'each' when you know the gap you want per element.
//        Use 'amount' when you know the total time you want.
```

■ **Grid Stagger — 2D Elements**

● ● ● *Grid stagger*

```
// Gallery ya grid layout ke liye
gsap.from('.grid-photo', {
  scale: 0,
  opacity: 0,
  duration: 0.4,
  ease: 'back.out(1.5)',
  stagger: {
    grid: 'auto', // GSAP columns count karega
    // ya explicitly: grid: [3, 4] // 3 rows, 4 cols
    from: 'center',
    each: 0.04
  }
});

// Wave effect on axis
gsap.to('.cell', {
  y: -20,
  yoyo: true,
  repeat: -1,
  stagger: { grid: 'auto', from: 0, each: 0.05, axis: 'x' }
});
```

■ **Real Examples**

● ● ● *Real-world stagger examples*

```
// 1. Nav links entrance
gsap.from('.nav a', { opacity: 0, y: -15, stagger: 0.08, duration: 0.5 });

// 2. Card grid reveal on scroll
ScrollTrigger.batch(".card", {
  onEnter: els => gsap.from(els, {
    opacity: 0, y: 60,
    stagger: { each: 0.1, from: 'start' },
    duration: 0.7
  })
});

// 3. Letter by letter text reveal (with SplitText plugin)
const split = new SplitText('.heading', { type: 'chars' });
gsap.from(split.chars, {
  opacity: 0, y: 20, rotation: -10,
  stagger: 0.03, duration: 0.5,
  ease: 'back.out(1.5)'
});

// 4. Percentage-based stagger timing
gsap.to('.progress-bar', {
  width: '100%',
  stagger: { amount: 2, from: 'end' },
  duration: 1.5,
  ease: 'power2.out'
```

Chapter 8 — ScrollTrigger ■ Scroll Magic

ScrollTrigger = Scroll position se animations trigger karo

User jab scroll karta hai, tab animations start/stop/reverse/scrub hoti hain.

Ye GSAP ka sabse popular plugin hai — modern web design ka backbone!

MUST DO FIRST: `gsap.registerPlugin(ScrollTrigger);`

■ REAL LIFE ANALOGY:

Phone ka camera: jab tum zoom karte ho, image sharper hoti hai.

ScrollTrigger: jab user scroll kare, animation trigger ho.

Scrub mode: scroll down = animation forward, scroll up = animation reverse!

Jaise film rewind/forward karna — scroll ke saath 1:1 linked.

■ ScrollTrigger — 3 Ways to Use

3 ways to use ScrollTrigger

```
// ■ WAY 1: Shorthand string (simplest)
gsap.to('.box', {
  x: 400,
  scrollTrigger: '.box' // jab .box viewport mein aaye, trigger!
});

// ■ WAY 2: scrollTrigger object in tween
gsap.to('.section', {
  opacity: 1,
  y: 0,
  scrollTrigger: {
    trigger: '.section', // kaunsa element monitor karo
    start: 'top 80%', // trigger ka top, viewport ka 80% pe
    end: 'bottom 20%', // kab end hoga
    toggleActions: 'play none none reverse',
  }
});

// ■ WAY 3: Standalone ScrollTrigger.create()
ScrollTrigger.create({
  trigger: '.panel',
  start: 'top top',
  end: '+=500',
  pin: true, // element ko scroll mein pin karo
  animation: myTimeline, // is timeline ko control karo
  scrub: 1,
  markers: true,
});
```

■ start / end — Position Syntax

start aur end strings hain: 'trigger_position viewport_position'

Trigger position = element ka kaunsa point | Viewport position = screen ka kaunsa point

start / end position syntax

```
// Position values: top, center, bottom, pixels, percentages

start: 'top 80%' // element ka top, viewport ka 80% pe aane pe trigger
start: 'top center' // element ka top, screen ke center pe
start: 'top top' // element ka top, screen ke top pe (full reveal pe trigger)
start: 'center center' // element ka center, screen ke center pe
start: 'bottom top' // element ka bottom, screen ke top par aane pe

// Pixel offset bhi de sakte ho:
start: 'top 80%-50' // 80% point se 50px upar
start: '+=100' // scroll 100px ke baad

end: '+=500' // start se 500px scroll aur
end: 'bottom bottom' // element ka bottom, screen ka bottom

// TIP: markers: true lagao aur visually dekho positions!
```

toggleActions — 4 State Control

toggleActions = 'onEnter onLeave onEnterBack onLeaveBack'

Har state pe kya karna hai ye batao:

```
toggleActions
// 4 trigger states:
// onEnter:     element scroll mein enter kare (top cross)
// onLeave:      element scroll se exit kare (bottom cross)
// onEnterBack: wapas upar scroll karo, element re-enter kare
// onLeaveBack:  wapas upar scroll karo, element ke upar jao

// Available actions: 'play' 'pause' 'resume' 'reverse' 'restart' 'complete' 'reset' 'none'

toggleActions: 'play none none none' // sirf ek baar play karo (default-like)
toggleActions: 'play none none reverse' // enter pe play, upar jaane pe reverse
toggleActions: 'play pause resume reverse' // most complete
toggleActions: 'restart none none reset' // har baar enter pe restart

// Example:
gsap.to('.el', {
  x: 200, opacity: 1,
  scrollTrigger: {
    trigger: '.el',
    start: 'top 75%',
    toggleActions: 'play none none reverse'
  }
});
```

scrub — Scroll Se Linked Animation

scrub = animation scroll position se 1:1 link hoti hai

scrub: true → Instant link. Scroll karo, animation saath chalti hai. No lag.
scrub: 1 → 1 second lag/smoothing. Smooth chase effect. (MOST USED)
scrub: 0.5 → 0.5s lag. Faster response.
scrub: 2 → 2s lag. Very smooth, dreamy feel.

```
scrub examples
// Parallax effect — image scroll se linked
gsap.to('.hero-bg', {
  yPercent: -30, // scroll karne pe 30% upar jaayega
  ease: 'none', // scrub ke saath ALWAYS ease: 'none' use karo!
  scrollTrigger: {
    trigger: '.hero',
    start: 'top top',
    end: 'bottom top',
    scrub: 1,
  }
});

// Timeline ko scroll se control karo
const tl = gsap.timeline({
  scrollTrigger: {
    trigger: '.section',
    start: 'top top',
    end: '+=1000',
    scrub: 1,
    pin: true, // section scroll ke dauraan pin rhega
  }
});
tl.to('.circle', { x: 400 })
  .to('.square', { rotation: 360 }, '<')
  .to('.text', { opacity: 0 });
```

pin — Element Ko Freeze Karo

```
pin + scrub = cinematic scroll
// Pin = element ko viewport mein fix karo jab tak scroll range na khatam ho

ScrollTrigger.create({
  trigger: '.sticky-section',
  start: 'top top',
  end: '+=800', // 800px scroll tak pinned rahega
  pin: true,
  pinSpacing: true, // pinned element ke liye space preserve karo (default true)
});
```


■ ScrollTrigger.batch() — List Items

```
● ● ● ScrollTrigger.batch()
// Jab bohot saare elements viewport mein aayein tab batch animate karo
// Individual ScrollTrigger se BAHUT better performance!

ScrollTrigger.batch('.card', {
  interval: 0.1, // elements ko group karo (0.1s window mein jo aayein)
  batchMax: 3, // ek baar mein max 3 cards animate karo
  onEnter: (batch) => {
    gsap.from(batch, {
      opacity: 0,
      y: 60,
      stagger: 0.1,
      duration: 0.8,
      ease: 'power3.out'
    })
  },
  onLeave: (batch) => gsap.set(batch, { opacity: 0 }),
  onEnterBack: (batch) => gsap.to(batch, { opacity: 1 }),
  onLeaveBack: (batch) => gsap.set(batch, { opacity: 0 }),
  start: 'top 85%',
  end: 'bottom 15%',
});
```

■ Other Important ScrollTrigger Options

markers: true

Start/End points visually dikho. ONLY use in development — production mein HATA DO!

toggleClass: 'active'

Trigger range mein element pe class add/remove karo. Pure CSS transitions ke liye!

once: true

Sirf ek baar trigger karo phir destroy. Entrance animations ke liye perfect!

onEnter / onLeave etc

Callbacks — trigger points pe custom code chalao. Accepts ScrollTrigger instance.

invalidateOnRefresh: true

Window resize pe tween ke start values recapture karo. Responsive ke liye zaroor!

horizontal: true

Horizontal scroll ke liye. Vertical behaviour same rehti hai otherwise.

containerAnimation

Horizontal scroll container ke andar use karo. Container tween pass karo.

pinnedContainer

Agar trigger kisi pinned element ke andar hai toh parent pin pass karo.

■ ScrollTrigger Static Methods

```
● ● ● ScrollTrigger utility methods

ScrollTrigger.refresh(); // layout change ke baad recalculate
ScrollTrigger.update(); // force update positions
ScrollTrigger.getAll(); // sab active ScrollTriggers ka array
ScrollTrigger.getById('myId'); // specific ST instance
ScrollTrigger.kill(reset); // saare ScrollTriggers kill karo
ScrollTrigger.killAll(); // sab kill + cleanup

// Smooth Scroller ke saath (Lenis, Locomotive etc)
ScrollTrigger.scrollerProxy('.wrapper', {
  scrollTop(value) { return arguments.length ? lenis.scrollTo(value) : lenis.scroll; },
  getBoundingClientRect() { return { top:0, left:0, width: window.innerWidth, height: window.in
});
// Phir ScrollTrigger ko Lenis ke saath sync karo:
lenis.on('scroll', ScrollTrigger.update);
```

Chapter 9 — Callbacks, Controls & gsap.utils ■■

■ Callbacks — Animation Events Pe Code Chalao

```
● ● ● Callbacks
gsap.to('.box', {
  x: 300, duration: 2,

  onStart:      () => console.log('Shuru!'),
  onUpdate:     () => console.log('Progress:', tween.progress()),
  onComplete:   () => console.log('Khatam!'),
  onRepeat:     () => console.log('Repeat!'),
  onReverseComplete: () => console.log('Reverse done!'),

  // Arguments pass karo callbacks mein:
  onCompleteParams: ['arg1', myObj], // array of args
  onStartParams:    ['hello'],

  // Ya modern way — arrow function mein directly use karo:
  onComplete: () => {
    modal.classList.add('done');
    nextAnimation();
  }
});

// TIMELINE callbacks:
const tl = gsap.timeline({
  onStart:    () => showLoader(),
  onComplete: () => hideLoader(),
  onUpdate:   () => updateProgressBar(tl.progress()),
});
```

■ gsap.delayedCall() — setTimeout ka GSAP Version

```
● ● ● gsap.delayedCall()
// Normal setTimeout — GSAP ke saath sync nahi hota (pause pe bhi chalta hai!)
setTimeout(() => doSomething(), 1000); // AVOID in GSAP apps

// gsap.delayedCall() — GSAP ke saath synced!
gsap.delayedCall(1, doSomething); // 1 second baad call karo
gsap.delayedCall(0.5, () => modal.show()); // 0.5s baad modal dikhao

// Arguments pass karo:
gsap.delayedCall(2, myFn, ['arg1', 'arg2']);

// Return value save karo — kill karna ho toh
const timer = gsap.delayedCall(3, cleanup);
// Baad mein cancel karo:
timer.kill();

// WHY better than setTimeout?
// GSAP pause karo => delayedCall bhi pause hoga
// timeScale change karo => delayedCall bhi scale hoga
// Perfect sync with your animations!
```

■ gsap.utils — Powerful Helper Functions

```
gsap.utils.clamp(min, max, value)
Value ko ek range mein rakho. Boundary cross nahi hogi.
gsap.utils.clamp(0, 100, 150) => 100
gsap.utils.clamp(0, 100, -20) => 0
// Reusable function banao:
const clamp0to100 = gsap.utils.clamp(0, 100);
clamp0to100(150); => 100 | clamp0to100(-5); => 0
```

gsap.utils.mapRange(in1, in2, out1, out2, value)

Ek range ki value ko doosri range mein map karo.

```
// Scroll 0-500px ko opacity 0-1 mein convert karo:
const mapScroll = gsap.utils.mapRange(0, 500, 0, 1);
mapScroll(250) => 0.5
// Real use: scroll progress to rotation
const toRotation = gsap.utils.mapRange(0, 1, 0, 360);
gsap.to('.wheel', { rotation: toRotation(scrollProgress) });
```

gsap.utils.interpolate(start, end, progress)

Do values ke beech koi bhi value nikalo (0-1 progress se).

```
gsap.utils.interpolate(0, 100, 0.5) => 50
gsap.utils.interpolate("red", "blue", 0.5) => "#7f007f" (color mix!)
gsap.utils.interpolate([1, 2, 3], 0.5) => 2 (array interpolation)
// Easing ke saath:
gsap.utils.interpolate(0, 100, 0.5, gsap.parseEase("power2"))
```

gsap.utils.wrap(values)

Array ya range ko wrap karo — cyclic/circular index.

```
const wrap = gsap.utils.wrap(['red', 'blue', 'green']);
wrap(0) => 'red' wrap(1) => 'blue'
wrap(3) => 'red' wrap(4) => 'blue' // wraps around!
// Number range wrap:
const wrapAngle = gsap.utils.wrap(0, 360);
wrapAngle(400) => 40 (360 ke baad 40)
```

gsap.utils.random(min, max, snap?)

Random number generate karo. Snap optional hai.

```
gsap.utils.random(0, 360) // random 0-360
gsap.utils.random(0, 360, 45) // 0, 45, 90, 135... mein snap karo
gsap.utils.random(['red', 'blue', 'green']) // random array item
// Reusable randomizer:
const rndPos = gsap.utils.random(-100, 100, true); // returns fn
rndPos() => e.g. 47 rndPos() => -83
```

gsap.utils.toArray(selector)

Kuch bhi pass karo — real Array milega. NodeList, string, element sab.

```
gsap.utils.toArray('.item') // NodeList => Array
gsap.utils.toArray('#hero') // single element => [element]
gsap.utils.toArray(myRef) // React ref => [element]
// Useful when you need array methods (.map, .filter) on elements
```

gsap.utils.pipe(...functions)

Multiple transform functions ko chain karo — output ek se doosre mein jaata hai.

```
const transform = gsap.utils.pipe(
  gsap.utils.clamp(0, 100),
  gsap.utils.mapRange(0, 100, 0, 360),
  Math.round
);
transform(50) => 180 transform(150) => 360
```

gsap.utils.unitize(fn, unit)

Function ko CSS unit-aware banao.

```
const clampPx = gsap.utils.unitize(gsap.utils.clamp(0, 500), 'px');
clampPx('200px') => '200px'
clampPx('600px') => '500px' // clamped!
```

gsap.ticker — Internal Animation Loop

● ● ● gsap.ticker & globalTimeline

// GSAP ka internal requestAnimationFrame loop — very powerful!

// Har frame pe kuch karo (like RAF but better):

```
gsap.ticker.add((time, deltaTime, frame) => {
  // time = seconds since GSAP start
  // deltaTime = time since last frame (ms)
  // frame = frame number
  myCustomLoop(deltaTime);
});
```

// Remove karo:

```
const myFn = () => { ... };
gsap.ticker.add(myFn);
gsap.ticker.remove(myFn);
```

// FPS set karo (default = display refresh rate):

```
gsap.ticker.fps(30); // 30fps pe limit karo (battery save)
```

// Sleep duration (how long GSAP waits after inactivity):

Chapter 10 — GSAP Plugins ■ Superpowers!

Plugins = GSAP ke special powers — extra features jo core mein nahi hain

FREE plugins: gsap npm package ke saath aate hain.

Club plugins: gsap.com membership chahiye (SplitText, DrawSVG, MorphSVG, etc.)

TextPlugin [FREE]

Text content ko character by character animate karo — typewriter effect!

```
gsap.registerPlugin(TextPlugin);

// Typewriter effect
gsap.to(".heading", {
  text: "Hello India! Kya haal hai?",
  duration: 2,
  ease: "none", // character-by-character evenly
});

// HTML tags bhi support karta hai:
gsap.to(".el", {
  text: { value: "<strong>Bold text</strong> animate!", type: "diff" },
  duration: 1.5
});

// type: 'diff' = sirf jo different hai woh animate karo (smart!)
```

Flip Plugin [FREE]

FLIP technique: layout changes ko smoothly animate karo. DOM change ho toh bhi!

```
gsap.registerPlugin(Flip);

// FLIP = First, Last, Invert, Play
// Step 1: Current state capture karo
const state = Flip.getState('.items');

// Step 2: DOM ya layout change karo
container.classList.toggle("grid-layout");
item.parentNode.appendChild(item); // ya move karo element ko

// Step 3: GSAP smoothly transition karega pehli se doosri jagah
Flip.from(state, {
  duration: 0.6,
  ease: "power1.inOut",
  stagger: 0.05,
  absolute: true, // absolute positioning use karo transition mein
  onComplete: () => console.log('Flip done!')
});

// Advanced: Nested elements ke saath
const state = Flip.getState(".card, .card *"); // children bhi!
```

Draggable [FREE]

Elements ko drag/drop karo — inertia, bounds, snap ke saath!

```
gsap.registerPlugin(Draggable, InertiaPlugin); // InertiaPlugin = Club

const drag = Draggable.create('.ball', {
  type: 'x,y', // x, y, xa (both), rotation
  bounds: '.container', // drag boundary
  inertia: true, // throw effect (needs InertiaPlugin)
  edgeResistance: 0.5, // boundary ke paas resistance
  dragResistance: 0, // drag friction
  snap: { x: 50 }, // 50px grid mein snap karo
  // ya snap function:
  snap: { x: val => Math.round(val / 50) * 50 },
  onClick: () => console.log('Click!'),
  onDragStart: function() { console.log('Drag start', this.x, this.y); },
  onDrag: function() { updatePosition(this.x, this.y); },
  onDragEnd: function() { checkDropZone(this.x, this.y); },
  onThrowComplete: () => console.log('Throw done!'),
  lockAxis: false, // drag shuru hone pe axis lock ho jaaye
  cursor: 'grab',
  activeCursor: 'grabbing',
  zIndexBoost: true // drag pe z index increase
```

Observer [FREE]

Touch, mouse wheel, pointer events uniformly detect karo — custom scroll!

```
gsap.registerPlugin(Observer);

Observer.create({
  target: window,
  type: "wheel,touch,pointer", // ya "scroll"
  onDown: () => goToPrevSection(),
  onUp: () => goToNextSection(),
  onLeft: () => previousSlide(),
  onRight: () => nextSlide(),
  tolerance: 10, // minimum movement to trigger
  preventDefault: true,
  onDrag: self => {
    // self.deltaX, self.deltaY, self.velocityX, self.velocityY
    updateDrag(self.deltaX);
  },
  // Used for: fullpage scroll, custom sliders, touch gestures!
});
```

SplitText [Club]

Text ko chars/words/lines mein split karo — phir har piece animate karo!

```
gsap.registerPlugin(SplitText);

// Text split karo
const split = new SplitText(".heading", {
  type: 'chars,words,lines', // ya sirf 'chars'
  charsClass: 'char',
  wordsClass: 'word',
  linesClass: 'line',
});

// split.chars = array of char elements
// split.words = array of word elements
// split.lines = array of line elements

// Animate chars ek ek karke
gsap.from(split.chars, {
  opacity: 0, y: 20, rotation: -10,
  stagger: 0.03, duration: 0.5,
  ease: 'back.out(1.5)'
});

// Cleanup — DOM restore karo
split.revert();
```

DrawSVG [Club]

SVG paths ko draw karo — signature writing, line drawing, reveal effects!

```
gsap.registerPlugin(DrawSVG);

// SVG path ko invisible se draw karo
gsap.from(".signature path", {
  drawSVG: "0%", // 0% se 100% tak draw hoga
  duration: 3,
  ease: "none"
});

// Partially draw karo:
gsap.to(".path", { drawSVG: "25% 75%", duration: 1 }); // middle 50% visible

// ScrollTrigger ke saath — scroll pe draw karo!
gsap.to(".road path", {
  drawSVG: "100%",
  ease: 'none',
  scrollTrigger: { trigger: '.road', scrub: 1 }
});
```

MorphSVG [Club]

Ek SVG shape se doosre mein morph karo — butter smooth!

```
gsap.registerPlugin(MorphSVGPlugin);

// Shape A se Shape B mein morph karo
gsap.to("#star", {
  morphSVG: "#circle", // target shape ka selector
  duration: 1,
  ease: "power2.inOut"
});

// String path bhi accept karta hai:
gsap.to("#shape", {
  morphSVG: "M10,10 L90,10 L90,90 Z", // raw path data
  duration: 0.8
});

// Multiple morphs sequence mein:
const tl = gsap.timeline({ repeat: -1, yoyo: true });
tl.to("#shape", { morphSVG: "#heart", duration: 1 })
  .to("#shape", { morphSVG: "#star", duration: 1 });
```

MotionPathPlugin [FREE]

Element ko SVG path pe ghoomao — curved motion paths!

```
gsap.registerPlugin(MotionPathPlugin);

// SVG path pe animate karo
gsap.to(".car", {
  duration: 5,
  ease: 'none',
  motionPath: {
    path: "#road", // SVG path element ka selector
    align: "#road", // path ke saath align
    autoRotate: true, // direction ke according rotate karo
    alignOrigin: [0.5, 0.5], // element ka center align point
    start: 0, // 0 = path ka start
    end: 1, // 1 = path ka end
  }
});

// Custom array of points bhi use kar sakte ho:
gsap.to(".ball", {
  motionPath: [
    { x: 100, y: 200 },
    { x: 300, y: 50 },
    { x: 500, y: 200 },
  ],
  duration: 2
});
```

CustomEase [FREE]

Apna custom Bezier curve ease banao — koi bhi feel!

```
gsap.registerPlugin(CustomEase);

// SVG-path-style Bezier curve string se ease create karo
CustomEase.create('hop', 'M0,0 C0.5,0 0.5,1 1,1');
CustomEase.create('myEase', 'M0,0 C0.14,0 0.242,0.438 0.372,0.561',
  + '0.566,0.749 0.704,1 1,1');

// Ab use karo:
gsap.to('.el', { x: 200, ease: 'hop' });

// GSAP Ease Visualizer pe curve draw karo aur code copy karo!
// gsap.com/ease-visualizer — best tool for this!

// gsap.parseEase() — string ko function mein convert karo
const fn = gsap.parseEase('power2.out');
fn(0.5) => 0.75 (progress value at 50%)
```

-
-
-
-
-
-
-
-
-
-
-
-
-
-
-
-
-

Chapter 11 — Performance & GPU Tricks

Animation smooth hogi tabhi jab GPU pe chalegi — CPU pe nahi.

GPU = Graphics Processing Unit — har frame draw karne ka kaam ye karta hai.

CPU animation = janky ■ | GPU animation = buttery smooth 60fps ■

■ GPU-Friendly Properties — ALWAYS USE THESE

GSAP Property	CSS Equivalent	Why GPU?
x, y	transform: translate()	Screen pe move karo — NO layout recalc
xPercent, yPercent	transform: translate(%)	Percentage-based move
rotation	transform: rotate()	Rotate karo
scale, scaleX, scaleY	transform: scale()	Size change
skewX, skewY	transform: skew()	Skew/slant
opacity	opacity	Visibility change
autoAlpha	opacity + visibility	Smart visibility
rotationX, rotationY	transform: perspective rotateX/Y	3D flip effects

■ **COMMON MISTAKE:**

NEVER animate: top, left, right, bottom — ye layout recalculate karta hai (SLOW!)

NEVER animate: width, height, margin, padding for performance animations.

INSTEAD: x/y use karo movement ke liye, scale use karo size change ke liye.

BAD: `gsap.to('.el', { left: 100 })` // layout thrash!

```
GOOD: gsap.to('.el', { x: 100 }) // GPU-only, smooth!
```

BAD: `gsap.to('.box', { width: 200 })` // layout recalc

```
GOOD: gsap.to('.box', { scaleX: 2 }) // GPU! (origin adjust karo if needed)
```

■ Memory Management — Leaks Se Bachhao

- ● ● *Memory management*

[illegible][illegible][illegible]

```
// Cleanup (React useEffect return):
return () => ctx.revert(); // SAARE animations, ScrollTriggers kill!
```

[illegible]

- **overwrite: 'auto' — Conflict Management**

● ● ● *overwrite best practice*

```
// PROBLEM: Rapid hover pe multiple conflicting tweens
btn.addEventListener('mouseenter', () => {
  gsap.to(btn, { scale: 1.2, duration: 0.2 }); // tween 1
});
btn.addEventListener('mouseenter', () => {
  gsap.to(btn, { scale: 1.2, duration: 0.2 }); // tween 2 - conflict!
});
// Dono saath chalenge - glitch!
```

```
// SOLUTION:
gsap.to(btn, { scale: 1.2, duration: 0.2, overwrite: 'auto' });
// 'auto' = sirf conflicting properties override karo
```


- x/y use karo, top/left nahi
- opacity animate karo, visibility nahi (ya autoAlpha)
- gsap.context() use karo React/Vue mein
- overwrite: 'auto' globally set karo
- will-change: transform; CSS mein add karo heavily animated elements pe
- ScrollTrigger.batch() use karo lists ke liye (vs individual ST per item)
- Cleanup karo: kill(), context.revert(), st.kill()
- markers: true sirf development mein, production mein hata do
- ease: 'none' use karo scrub animations mein
- gsap.set() use karo initial states ke liye — FOUC prevent karo
- width/height animate karna — sirf zaroorat ho toh karo
- filter animate karna (blur, brightness) — can be slow on mobile
- Very long timelines with 100s of tweens — consider chunking
- left/top animate karna — NEVER (layout thrash)
- margin/padding animate karna — NEVER for performance animations
- Cleanup nahi karna React components mein — memory leak!
- markers: true production mein — shows ugly debug overlays

Chapter 12 — React & Vue Integration ■■

React/Vue ke saath GSAP = Perfect combo lekin kuch rules follow karne padte hain

React = components mount/unmount karta hai — GSAP cleanup zaroor karo!

gsap.context() = React-friendly GSAP wrapper — isko use karo hamesha.

■ React — useEffect + gsap.context()

●●● React useEffect + gsap.context()

```
import { useEffect, useRef } from 'react';
import { gsap } from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';

gsap.registerPlugin(ScrollTrigger); // App.js mein ek baar!

function MyComponent() {
  const containerRef = useRef(null);

  useEffect(() => {
    // gsap.context() use karo — scope + cleanup!
    const ctx = gsap.context(() => {

      // Yahan GSAP code likho — selector sirf containerRef ke andar dhundha jaayega
      gsap.from('.title', { opacity: 0, y: 30, duration: 1 });
      gsap.from('.card', { opacity: 0, stagger: 0.1 });

      ScrollTrigger.create({
        trigger: '.section',
        start: 'top 80%',
        onEnter: () => gsap.to('.section', { opacity: 1 })
      });

    }, containerRef); // ← scope element

    // Cleanup function — component unmount pe sab kill hoga
    return () => ctx.revert();

  }, []); // [] = sirf mount pe run karo

  return (
    <div ref={containerRef}>
      <h1 className='title'>Hello!</h1>
      <div className='card'>Card 1</div>
      <div className='card'>Card 2</div>
    </div>
  );
}
```

■ Custom Hook — useGSAP() from @gsap/react

●●● @gsap/react — useGSAP hook

// GSAP ki official React library — useEffect wrapper!
npm install gsap @gsap/react

```
import { useGSAP } from '@gsap/react';
import { gsap } from 'gsap';

gsap.registerPlugin(useGSAP); // App.js mein register karo

function MyComponent() {
  const containerRef = useRef(null);

  // useGSAP = useEffect + gsap.context() combo!
  useGSAP(() => {
    gsap.from('.box', { x: -100, opacity: 0, duration: 1 });

    // ScrollTrigger bhi yahan:
    gsap.to('.section', {
      opacity: 1,
      scrollTrigger: { trigger: '.section', start: 'top 80%' }
    });

    // Cleanup automatic hai — no need for return!
  }, { scope: containerRef }); // scope = container

  // Dependencies (re-run kab karo):
  useGSAP(() => {
```

■ Ref-Based Animation — Direct Element Access

```
● ● ● Ref-based animation
// Jab tumhe specific element pe animate karna ho (ref ke through)
function AnimatedButton() {
  const btnRef = useRef(null);

  useGSAP(() => {
    const tween = gsap.to(btnRef.current, {
      scale: 1.1,
      duration: 0.2,
      paused: true // paused rakhna pehle
    });

    btnRef.current.addEventListener('mouseenter', () => tween.play());
    btnRef.current.addEventListener('mouseleave', () => tween.reverse());

  }, { scope: btnRef });

  return <button ref={btnRef}>Hover Me!</button>;
}
```

■ Vue 3 — onMounted + onUnmounted

```
● ● ● Vue 3 — gsap.context() integration
<template>
  <div ref='container'>
    <h1 class='title'>Hello Vue!</h1>
    <div class='card' v-for='i in 3' :key='i'>Card {{ i }}</div>
  </div>
</template>

<script setup>
import { ref, onMounted, onUnmounted } from 'vue';
import { gsap } from 'gsap';

const container = ref(null);
let ctx; // context save karo

onMounted(() => {
  ctx = gsap.context(() => {
    gsap.from('.title', { opacity: 0, y: 30, duration: 1 });
    gsap.from('.card', { opacity: 0, x: -30, stagger: 0.15, duration: 0.6 });
  }, container.value); // scope = template ref
});

onUnmounted(() => {
  ctx.revert(); // cleanup!
});
</script>
```

■ State-Driven Animation — React State Se Animate Karo

```
● ● ● State-driven animations
// Animation jo React state ke saath sync ho
function ProgressBar({ percentage }) {
  const barRef = useRef(null);

  useGSAP(() => {
    gsap.to(barRef.current, {
      width: `${percentage}%`,
      duration: 0.8,
      ease: 'power2.out'
    });
  }, { dependencies: [percentage] }); // percentage change pe re-run

  return (
    <div className='bar-track'>
      <div ref={barRef} className='bar' style={{ width: '0%' }} />
    </div>
  );
}

// Page transition animation
function PageWrapper({ children, route }) {
  const pageRef = useRef(null);
```

Chapter 13 — Real-World Projects ■

PROJECT 1: Animated Landing Page Hero

```
// Complete hero section animation sequence
// HTML:
// <nav><a class='nav-link'>Home</a>...</nav>
// <section class='hero'>
//   <h1 class='hero-title'>Hello World</h1>
//   <p class='hero-sub'>Tagline here</p>
//   <button class='cta'>Get Started</button>
//   <img class='hero-img' src='...' />
// </section>

// Initial hide (FOUC prevent karo)
gsap.set(['.nav-link', '.hero-title', '.hero-sub', '.cta', '.hero-img'], {
  autoAlpha: 0
});

const intro = gsap.timeline({
  defaults: { ease: 'power3.out', duration: 0.8 },
  delay: 0.2
});

intro
  .from('.logo', { y: -20, autoAlpha: 0, duration: 0.5 })
  .to('.nav-link', { autoAlpha: 1, y: 0, stagger: 0.08 }, '<+0.1')
  .fromTo('.hero-title', { y: 60, autoAlpha: 0 }, { y: 0, autoAlpha: 1, duration: 1 }, '+=0.1')
  .from('.hero-sub', { y: 30, autoAlpha: 0 }, '<+0.3')
  .from('.cta', { scale: 0.8, autoAlpha: 0, ease: 'back.out(2)' }, '<+0.2')
  .from('.hero-img', { x: 80, autoAlpha: 0, duration: 1.2 }, 0.4);
// hero-img starts at 0.4s (absolute) – overlaps with text nicely!
```

PROJECT 2: Scroll-Reveal Card Grid

```
// Jab cards viewport mein aayein tab reveal karo
gsap.registerPlugin(ScrollTrigger);

// Initial state
gsap.set('.card', { autoAlpha: 0, y: 50 });

// Batch – performance-optimized!
ScrollTrigger.batch('.card', {
  interval: 0.15, // 0.15s window mein jo cards aayein unhe group karo
  batchMax: 4, // ek baar mein max 4 cards

  onEnter: (batch) =>
    gsap.to(batch, {
      autoAlpha: 1,
      y: 0,
      stagger: 0.1,
      duration: 0.7,
      ease: 'power2.out',
      overwrite: true
    }),

  onLeave: (batch) => gsap.set(batch, { autoAlpha: 0, y: -20 }),
  onEnterBack: (batch) =>
    gsap.to(batch, { autoAlpha: 1, y: 0, stagger: 0.1 }),
  onLeaveBack: (batch) => gsap.set(batch, { autoAlpha: 0, y: 50 }),

  start: 'top 85%',
  end: 'bottom 15%'
});

// Resize pe refresh karo
ScrollTrigger.addEventListener('refreshInit', () =>
  gsap.set('.card', { autoAlpha: 0, y: 50 })
);
```

PROJECT 3: Cinematic Scroll Story

```
// Section pin karo, scroll karte karte story reveal hogi
gsap.registerPlugin(ScrollTrigger);

// Story sections HTML:
// <div class='story-wrapper'>
//   <div class='scene'><img/><h2 class='title-1'/>...</div>
// </div>

const storyTL = gsap.timeline();

storyTL
  .from('.title-1', { opacity: 0, y: 40, duration: 1 })
  .from('.desc-1', { opacity: 0, duration: 0.8 }, '<+0.3')
  .to('.title-1', { opacity: 0, y: -30, duration: 0.8 }, '<+=0.5')
  .to('.desc-1', { opacity: 0 }, '<')
  .from('.img-2', { scale: 1.3, opacity: 0, duration: 1 })
  .from('.title-2', { opacity: 0, x: -50, duration: 0.8 }, '<+0.5');

ScrollTrigger.create({
  animation: storyTL,
  trigger: '.story-wrapper',
  start: 'top top',
  end: '+=3000', // 3000px scroll mein poori story
  pin: true,
  scrub: 1.5, // smooth scroll-linked
  anticipatePin: 1
});
```

PROJECT 4: Magnetic Cursor Button

```
// Button cursor ke paas magnetic attract hota hai
const btn = document.querySelector('.magnetic-btn');

btn.addEventListener('mousemove', (e) => {
  const rect = btn.getBoundingClientRect();
  const centerX = rect.left + rect.width / 2;
  const centerY = rect.top + rect.height / 2;

  // Cursor distance from center
  const deltaX = e.clientX - centerX;
  const deltaY = e.clientY - centerY;

  // Button thoda cursor ki taraf kheenche (20% attraction)
  gsap.to(btn, {
    x: deltaX * 0.2,
    y: deltaY * 0.2,
    duration: 0.5,
    ease: 'power2.out',
    overwrite: 'auto'
  });
});

btn.addEventListener('mouseleave', () => {
  gsap.to(btn, { x: 0, y: 0, duration: 0.7, ease: 'elastic.out(1, 0.3)' });
});
```

PROJECT 5: Counter Animation

```
// Scroll pe number counter 0 se target tak animate hoga
function animateCounter(element, target, duration = 2) {
  const obj = { val: 0 };
  const tween = gsap.to(obj, {
    val: target,
    duration: duration,
    ease: 'power2.out',
    onUpdate: () => {
      element.textContent = Math.round(obj.val).toLocaleString();
    },
    scrollTrigger: {
      trigger: element,
      start: 'top 80%',
      once: true // sirf ek baar!
    }
  });
}
```

PROJECT 6: SVG Path Draw on Scroll

```
// Scroll karne pe SVG path draw hoti jaayegi (Club plugin: DrawSVG)
gsap.registerPlugin(ScrollTrigger, DrawSVG);

// HTML: <svg><path class='road' d='M0,100 Q250,0 500,100...' /></svg>

// Initially path invisible
gsap.set('.road', { drawSVG: '0%' });

// Scroll ke saath draw karo
gsap.to('.road', {
  drawSVG: '100%',
  ease: 'none',
  scrollTrigger: {
    trigger: '.svg-section',
    start: 'top center',
    end: 'bottom center',
    scrub: 1,
    markers: false
  }
});

// Car path pe chalao (MotionPath):
gsap.to('.car-icon', {
  motionPath: {
    path: '.road',
    align: '.road',
    autoRotate: true,
    alignOrigin: [0.5, 0.5]
  },
  ease: 'none',
  scrollTrigger: { trigger: '.svg-section', scrub: 1,
    start: 'top center', end: 'bottom center' }
});
```

Practice Exercises

Level: Beginner

1. Ek box banao jo page load pe left se slide in kare (gsap.from)
2. Loading spinner banao jo infinite rotate kare
3. 3 cards banao jo stagger se fade in hon
4. Button hover pe scale up/down animation
5. Number counter 0 se 100 tak animate karo

Level: Intermediate

1. Timeline se page intro animation — logo, nav, hero text sequence
2. ScrollTrigger se cards reveal karo jab viewport mein aayein
3. Parallax hero section — image aur text alag speed se scroll karo
4. Accordion jo smoothly open/close ho (height animate karo)
5. Draggable slider — mouse se drag karke items reorder karo

Level: Advanced

1. Pinned scroll story — 3 scenes scrub ke saath
2. Magnetic cursor effect — mouse ke paas elements attract hon
3. SplitText se heading reveal — chars stagger se aayein
4. FLIP animation — grid to list layout smooth transition
5. SVG path draw on scroll + element path pe follow kare

■ Congratulations! GSAP Master Ban Gaye!

Ab practice karo, build karo, aur portfolio mein dalo! ■

gsap.com/docs · gsap.com/ease-visualizer · gsap.com/community